

Lab0 实验报告

姓名：杨梦琪

学号：25300120072

任务 1:

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我之前并没有多人协同开发的经历，但我很乐意在此后的学习中尝试。

2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

我认为 Git 设计“暂存-提交”的原因如下：

在代码设计的过程中，可能会出现各种各样的问题，于是我们需要针对这些问题进行改动，而改动则意味着原代码被替代；暂存则解决了这样的问题，每次保存的改动都是完整独立的，方便我们回撤和查看之前内容。

3. `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

`git branch`: 只查看本地创建的分支，而看不到远程仓库的分支。

`git branch -a`: 我查阅资料得到，“-a”表示全部 all，既能看到本地分支，也能看到远程仓库的分支，远程分支名字前面会带有“remotes/origin/”标识。

任务 2:

已创建个人仓库

任务 3:

1. Commit Message 规范:

内容概括:

(1) 为何需要明确规范 Commit Message

① 浏览提交历史时能够快速读懂每一次改动目的;

② 可以筛选特定类型提交（例如只看新增功能）；

（2）Commit message **格式** 格式的三大主要成分：Header（必填）、Body（可选，详细描述）、Footer（可选，标记不兼容变更、关闭 issue）。

（3）辅助工具

① Commitizen：可以辅助快速生成符合规范的提交信息；

② validate-commit-msg：检查提交信息格式，如果不合规则将阻止提交。

（4）Change log 自动生成：当所有提交格式均规范，Change log 即可自动生成生成的文档包含：New features、Bug fixes、Breaking changes

2. Git Flow 分支控制：

内容概括：

Git flow 是一套 Git 分支管理的工作流程，适用于规范多人协作的情况来避免版本混乱。其中共分为 2 个永久长期分支和 3 类临时分支。

（1）永久分支

① **master**：对应线上生产环境，保存稳定已发布代码；只接受别的分支合并过来，不直接在上面写代码；每次上线后打版本 tag 标记发布点。

② **develop**：公共开发分支，从 master 分出；所有新功能最终合并到此，存放待发布的开发代码。

（2）临时分支（需要在用完后删除）

① **feature（功能分支）**：从 develop 创建，开发单个新功能；开发完合并回 develop 然后删除分支。

② **release（预发布分支）**：全部功能开发完毕，从 develop 分出；用于版本测试、修改版本号、修复测试 bug；测试完成同时合并到 master 与 develop，打 tag，之后删除 release 分支。

③ **hotfix（热修复分支）**：线上出现紧急 bug，直接从 master 创建；修复线上问题，修复完成同时合并进 master 和 develop，删除该分支。

（3）操作注意事项

① 合并可以使用 `git merge --no-ff`，保留分支历史记录；

② master 分支上打版本 tag，记录发布版本；分支命名推荐使用短横线分隔。

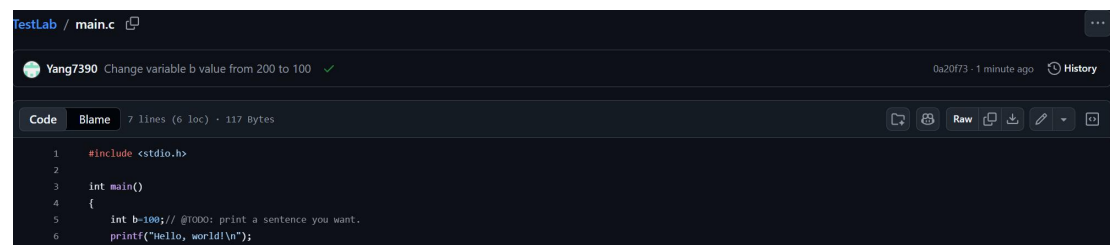
3. 我对于为什么要学习 git 的看法：

(1) 跟踪代码修改，使旧代码不易丢失：我之前在学习过程中制作网站，只使用了 VS code，制作过程中发现了很多问题，一旦我关闭电脑想要第二天再来完成任务，出现了无法撤销的情况；或是修改代码中的一些错误，造成了原来正确的地方出现问题，但是却因没有保存旧代码又需要一一检查，浪费大量时间。

(2) 方便团队协作：与单独使用 VS code 不同，既可以保证团队成员都参与进代码制作，又能辅助处理不同人之间的代码冲突，很适合之后进行实践和科研中应用。

任务 4.1：学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交；

实验过程中，我遇到了一些问题，我在 main 和 feature 两个分支中，依次修改并提交，main 中修改为：b=100



```
testLab / main.c
Yang7390 Change variable b value from 200 to 100 ✓ 0a20f73 · 1 minute ago History
Code Blame 7 lines (6 loc) · 117 Bytes
1 #include <stdio.h>
2
3 int main()
4 {
5     int b=100; // @TODO: print a sentence you want.
6     printf("Hello, world!\n");
7 }
```

feature 中修改为：a=200



```
testLab / feature
Yang7390 Change variable a value from 100 to 200 ✓ 0a20f73 · 1 minute ago History
Code Blame 7 lines (6 loc) · 117 Bytes
1 #include <stdio.h>
2
3 int main()
4 {
5     int a=200; // @TODO: print a sentence you want.
6     printf("Hello, world!\n");
7 }
```

任务 4.2：随后将 feature 分支 merge 到 main 分支（即切换回 main 分支执行 git merge feature），并处理发生的合并冲突。

我尝试操作合并，但尝试数次后出现了以下问题：

```
root@ics:~/TestLab# git checkout main
git pull origin main
git fetch origin feature
git merge feature
Already on 'main'
Your branch is up to date with 'origin/main'.
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 950 bytes | 475.00 KiB/s, done.
From github.com:Yang7390/TestLab
 * branch      main      -> FETCH_HEAD
    e23a476..1cfe9a8  main      -> origin/main
Updating e23a476..1cfe9a8
Fast-forward
 main.c | 2 +
 1 file changed, 1 insertion(+), 1 deletion(-)
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 1.93 KiB | 493.00 KiB/s, done.
From github.com:Yang7390/TestLab
 * branch      feature    -> FETCH_HEAD
 + 9054d0a...0a20f73  feature    -> origin/feature (forced update)
Already up to date.
root@ics:~/TestLab#
```

文件中的操作是正确的，但是却没有产生冲突；

我尝试通过克隆的方式，成功实现了冲突

```
root@ics: ~/TestLab/TestLab
* 28ffe13 Initial commit
root@ics:~/TestLab# git merge feature
Already up to date.
root@ics:~/TestLab# git clone git@github.com:Yang7390/TestLab.git
Cloning into 'TestLab'...
remote: Enumerating objects: 35, done.
remote: Counting objects: 100% (35/35), done.
remote: Compressing objects: 100% (33/33), done.
remote: Total 35 (delta 17), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (35/35), 11.68 KiB | 11.68 MiB/s, done.
Resolving deltas: 100% (17/17), done.
root@ics:~/TestLab# cd TestLab
root@ics:~/TestLab/TestLab# git log --oneline --graph --all -20
* 1cfe9a8 (HEAD -> main, origin/main, origin/HEAD) C
  change variable 'a' value from 100 to 200
* e23a476 modify main.c in main
  * 0a20f73 (origin/feature) Change variable b value from 200 to 100
  * 6e0bca6 Fix comment and declare variable in main.c
/
* 420c03a Replace comment with a numeric value in main.c
* 7089f17 main second modify same line
* 60920f0 change comment in main v2
* 4331683 modify main.c in main
* 2fcb047 Update TODO comment in main.c
* 28ffe13 Initial commit
root@ics:~/TestLab/TestLab# git merge origin/feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
root@ics:~/TestLab/TestLab#
```

所以本身的修改一定是没有问题的。接下来，我对日志图进行检验后，我发现问题出现在并没有及时更新 feature 而出现了这一系列问题

```
root@ics:~/TestLab# git log --oneline --graph --all -20
* 1cfe9a8 (HEAD -> main, origin/main, origin/HEAD) C
hange variable 'a' value from 100 to 200
* e23a476 modify main.c in main
| * 0a20f73 (origin/feature) Change variable b value from 200 to 100
| * 6e0bca6 Fix comment and declare variable in main.c
|/
* 420c03a Replace comment with a numeric value in main.c
* 7089f17 main second modify same line
* 60920f0 change comment in main v2
* 4331683 modify main.c in main
* 2fcb047 (feature) Update TODO comment in main.c
* 28ffe13 Initial commit
```

有代码中 0a20f73 这一行可见，feature 并没有得到及时的更新，导致提交没有同步

接下来执行 `git switch feature` 切换到 feature 分支，`git pull` 同步远程仓库的更新；再执行 `git switch main` 切回 main 分支，我成功在原仓库实现克隆

```
root@ics: ~/TestLab
root@ics:~/TestLab# ①切换到 feature 分支
git switch feature
# ②拉取远程feature的最新代码，同步本地feature
git pull
# ③切回 main 分支
git switch main
# ④合并本地feature，制造冲突
git merge feature
Switched to branch 'feature'
Your branch is behind 'origin/feature' by 6 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 1.84 KiB | 470.00 KiB/s, done.
From github.com:Yang7390/TestLab
 0a20f73..f7db148 feature -> origin/feature
 1cfe9a8..f2b2e5b main -> origin/main
Updating 2fcb047..f7db148
Fast-forward
 main.c | 2 +--
 1 file changed, 1 insertion(+), 1 deletion(-)
Switched to branch 'main'
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
root@ics:~/TestLab#
```

下一步，利用 nano 编辑进行修复

```
root@ics: ~/TestLab
From github.com:Yang7390/TestLab
0a20f73..f7db148 feature -> origin/feature
1cfe9a8..f2b2e5b main -> origin/main
Updating 2fcb047..f7db148
Fast-forward
 main.c | 2 +--
 1 file changed, 1 insertion(+), 1 deletion(-)
Switched to branch 'main'
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
root@ics:~/TestLab# cat main.c
#include <stdio.h>

int main()
{
<<<<<< HEAD
    int a=200;// @TODO: print a sentence you want.
=====
    int b=300;// @TODO: print a sentence you want.
>>>>>> feature
    printf("Hello, world!\n");
}
root@ics:~/TestLab# nano main.c
root@ics:~/TestLab# git add main.c
root@ics:~/TestLab# git commit
[main 1aeda5d] Merge branch 'feature'
root@ics:~/TestLab#
```

```
GNU nano 7.2 /root/TestLab/.git/COMMIT_EDITMSG
Merge branch 'feature'

# Conflicts:
#   main.c
#
# It looks like you may be committing a merge.
# If this is not correct, please run
#   git update-ref -d MERGE_HEAD
# and try again.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
#   (use "git pull" to update your local branch)
#
# All conflicts fixed but you are still merging.
#
# Untracked files:
#   TestLab/
#

[ Read 23 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-G Copy
```

已完成所有实验内容。